

AI for Economic Research: Dynamic Models, Language, and Agents

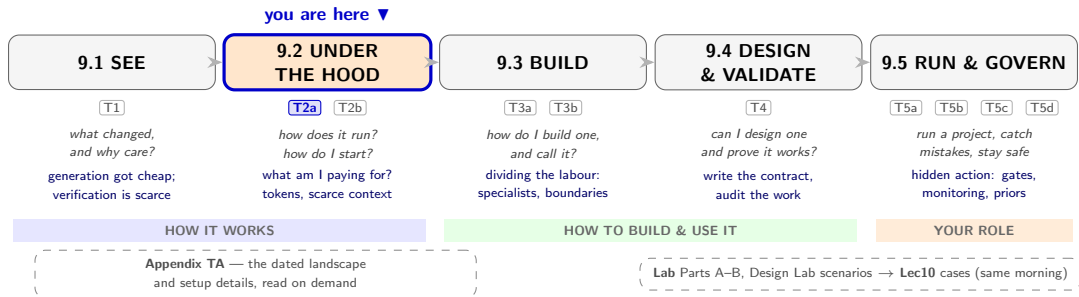
Lecture 9.2a: How an Agent Actually Runs — Harness, Session, Loop

Zhigang Feng

2026-09-12 20:25:56

Lecture 9 in One Map: How an Agent Runs

This part answers: how does an agent actually run — and how do I start one on my own machine?



Route: three components and one JSON line → the session is a list → one research question, traced from a real run. Three widespread beliefs turn out to be wrong on the way (T3b collects the corrections). The economist's question for this part: what exactly am I paying for?

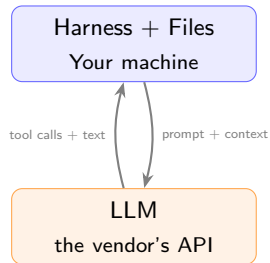
Three Components and One JSON Line

Harness, model, files — and the loop that joins them

Three Components: Harness, LLM, Your Files

A terminal agent has three physically distinct parts — the same anatomy in Claude Code, Codex CLI, Gemini CLI, or Aider:

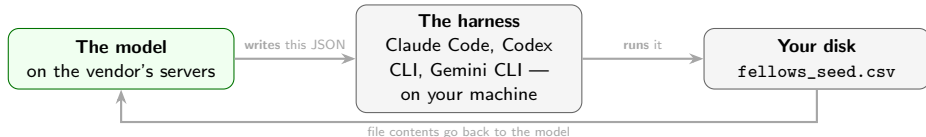
1. **The Harness** (lives on your machine)
CLI software you install. Reads your files, runs shell commands, manages the conversation loop.
2. **The LLM** (lives on the provider's servers)
Every message is bundled with conversation history and sent via HTTPS to the API. The “thinking” happens in the cloud.
3. **Your Files** (live on your machine)
Python scripts, data CSVs, LaTeX—all on your disk. Code executes on your hardware with your packages.



Key: code can run locally and many project files stay on your machine. That does *not* mean the whole workflow is automatically private.

From One JSON Line to a Running Loop

When an agent “reads a file,” the model does not read it. It writes a request — one line of structured JSON — and the harness does the reading.



What the model emits (schematic — field names differ by vendor):

```
{"tool": "Read", "args": {"file_path":  
"fellows_seed.csv"}}
```

- **The loop:** JSON out → the harness runs it → the result comes back as the next message → the next turn, until the model says it is done
- **The schema is the contract:** each tool advertises its arguments (Lec07 T4)
- **Same brain:** no separate planner — the model that writes code also decides to call tools

The naive mental model. “I open a session, the model remembers me, and somewhere a dispatcher assigns work to specialists.”

Every clause is wrong. This deck replaces it — and the cost model, the context limits, and the sub-agent behaviour of T3b and T5b–T5c all follow.

Privacy Boundary: Local Execution Is Not the Same as Local Processing

- code may execute locally on your machine
- many project files may stay on disk locally
- if file contents, prompts, or tool outputs are sent back in the conversation, those contents may still traverse the network
- only a fully local or institutionally private deployment changes that boundary

Practical rule: running code locally does not make it safe to put restricted data into the conversation — treat everything you type or attach as leaving your machine.

The Session Is a List

What a session is, and the loop that drives it

First, the Vocabulary: Who Is Speaking?

Every message carries a `role` label saying who produced it. There are three things you will see, and one is misleadingly named.

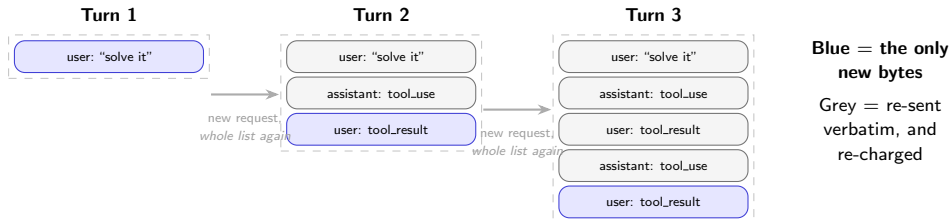
Label	Who actually wrote it	Example
"user"	you	"Solve the model and report r ."
"assistant"	the LLM itself	the JSON tool request on the last slide
"user", containing a <code>tool_result</code>	your harness, reporting what a tool returned	the contents of <code>fellows_seed.csv</code>

The naming trap. "Assistant" is simply the API's word for *the model's own turn* — it is not a separate helper program. And tool results are labelled "user" even though you did not type them: the API has only two speakers, so anything sent to the model is labelled "user", whoever produced it. Read "user" as "input to the model", not as "the human".

Why it matters: assistant means *the model*; tool output re-enters as user. That is all the vocabulary this deck needs.

The Session Is a List, Not a Connection

The API is stateless. Each turn is a separate request that carries the entire conversation so far — the model remembers nothing between calls.



- **"Re-send everything" is literal:** turn 3 is a fresh request carrying turns 1–3 in full
- No open connection, no server-side memory of you
- "Context window full" means *this list* outgrew the limit
- You are billed for the whole list each turn

Where the list starts and stops. /clear does exactly one thing: it throws the list away and starts an empty one. Compaction replaces the list with a summary of itself. Both are list operations — there is no "connection" to reset.

Four Moving Parts, One Function Call

An agent is not a service you connect to. It is four arguments you assemble and send.

Argument	What it is	Concretely
model	which LLM to send to	the string "claude-opus-5" — just a name; you are picking a brain
system	the standing instructions : role, rules, style, read <i>before</i> the conversation	"You are a computational macroeconomist. Never invent data." This is what CLAUDE.md becomes (T2b, T5b)
tools	the menu of actions — name, description, argument schema. <i>Descriptions only; no code is sent</i>	"Read: read a file. Arguments: file_path (string)." The model picks from this menu
messages	the conversation so far	every user / assistant / tool_result turn, in order

What comes back: ordinary text, or tool_use blocks — the model asking you to run something from the menu.

Change system and tools and you have a *different agent*; there is no "agent object" to create or shut down.

The Loop in Twenty Lines — Written by You, Once

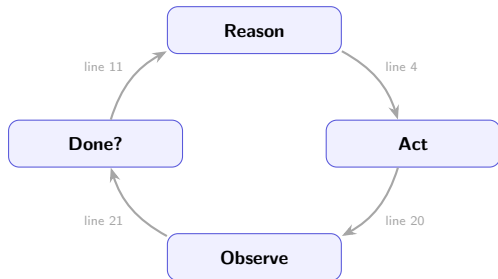
You write this loop once. The model never writes or runs it — it only supplies the replies the loop reads.

```
1 messages = [{"role": "user", "content": "Solve the model and report r."}]
2
3 while True:
4     resp = client.messages.create(
5         model="claude-opus-5",
6         max_tokens=16000,
7         system="You are a computational macroeconomist.",
8         tools=TOOLS,                # schemas, not functions
9         messages=messages,          # the whole list, every time
10    )
11    if resp.stop_reason != "tool_use":    # model is done talking
12        break
13    messages.append({"role": "assistant", "content": resp.content})
14
15    results = []
16    for b in resp.content:                # may be SEVERAL tool calls
17        if b.type == "tool_use":
18            results.append({"type": "tool_result",
19                            "tool_use_id": b.id,        # must match
20                            "content": run_tool(b.name, b.input)})
21    messages.append({"role": "user", "content": results})
```

Not automatic: this file — a person wrote it. **Automatic:** *which* tool, *which* arguments, when to stop — chosen by the model inside `create()`, line 4. **Line 9 with line 21:** the list only grows and is re-sent whole every pass.

ReAct, Named — and Where the Textbook Diagram Is Now Dated

The loop you just read has a name in the literature: **ReAct** — reason, act, observe, repeat (Yao et al., 2022).

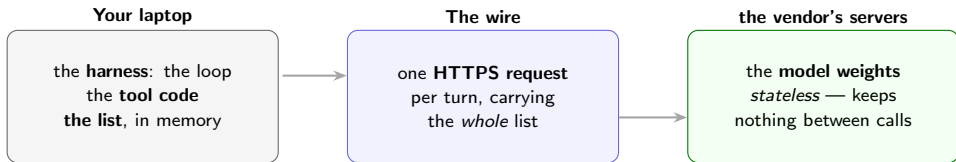


- **Reason** = the model's turn (line 4)
- **Act** = your harness runs the tool (line 20)
- **Observe** = the `tool_result` goes back (line 21)
- **Done?** = the `stop_reason` test (line 11)

Where the 2022 picture is dated. The original ReAct prompted the model to emit literal `Thought: / Action:` text that a parser scraped. Current models emit *structured* `tool_use` blocks and interleave reasoning with tool calls rather than strictly alternating. **Treat ReAct as the ancestor of the loop, not as its wire format** — if you implement the 2022 string-parsing version today you are rebuilding something the API already gives you typed.

So Where Does the Agent Live?

The JSON frame said: “the agent reads it and decides the next action.” That is convenient shorthand. Taken literally it is misleading — no single place contains the agent.



The agent is the loop, not a location. “The agent” names a *behaviour* that appears when a stateless model is driven by a stateful loop. Delete the loop and there is no agent — only a model that answers one question. Read T2’s sentence as: *the harness appended the tool result to the list, and the next request carried that list to the model.*

Why this is the practical frame: it tells you where each cost and limit lives — tokens and the context window on the *wire and the server*, tool permissions and your data on *your laptop* (T2’s privacy boundary), and the memory in a list *you* own and can clear.

Worked Example — GDP Growth by Party

One real research question, traced message by message

The Task — and the Trap Inside It

One prompt, typed once:

“Get US real GDP growth since 2000. Get which party held the White House over the same period. Then plot and rationalize how party affiliation affects GDP growth.”

Four jobs, not one

1. fetch a macro series (*a fact*)
2. build a party timeline (*a fact*)
3. plot it (*mechanical*)
4. explain the relationship (*a causal claim*)

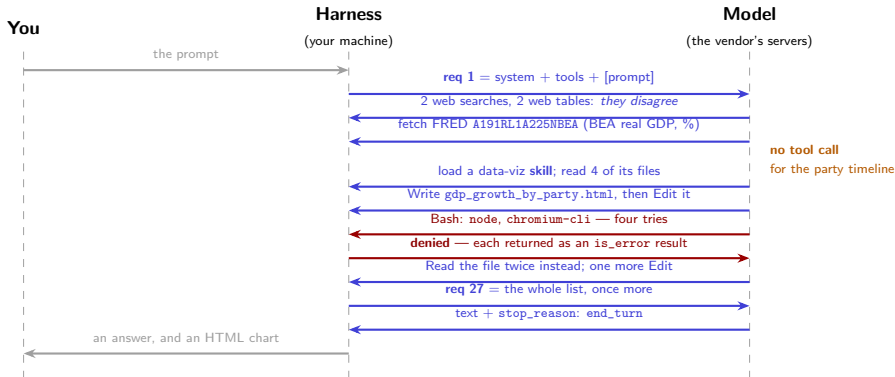
Jobs 1–3 an agent can do. Job 4 is where the research is — and where the prompt is booby-trapped.

“Rationalize” is a leading verb. It presupposes an effect and asks for a justification. We ran this exact prompt on a terminal agent in September 2026 to see what it would do with the verb — and where the weak points really were.

A chat model with no tools fails this invisibly: it recites GDP numbers from memory. They look right; some will be wrong; none are checkable.

One Agent, One Real Run: What Crossed the Wire

Twenty-seven requests, twenty-four tool calls, 280 seconds — and every request re-sent the whole list built so far.



Blue is billed; grey stays local. [solid] Recorded 12 Sep 2026 — Claude Code 2.1.270, claude-sonnet-5, \$0.76; trace in the lab's transcripts/; Part A, Step 4b replays it offline.

What Went Into req 1 — and Never Changed

This is T2b's initialization, seen from the wire: those five steps exist to build exactly two arguments — then re-sent byte-identical on every later request.

	Assembled from	In this example
system	T2b's init steps 1–4: global settings, CLAUDE.md, .claude/rules/*.md	"You are Claude Code. You have these tools..." + whatever house rules T5b has you write
tools	the schema of every tool the harness is willing to run	Bash, Read, Write, Grep... names, descriptions, argument types — <i>no code</i>

Yes — the model writes the code and asks your machine to run it. A Bash call is the model generating the *text* of a shell command as an argument. It executes nothing; your harness runs that string on your laptop, with your files and permissions.

The safety boundary is not the model's judgement — it is T2b's permission dial, between the generated string and your shell.

system is the only text still guaranteed to be there on turn 200 — everything else drifts or compacts away. That is the mechanical reason CLAUDE.md earns its keep.

Reading the Real Trace: Five Things Happened

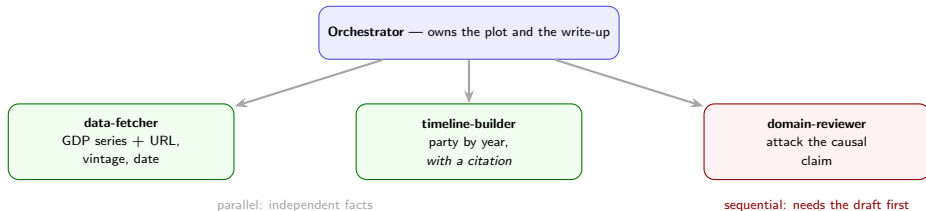
Every claim of the session section shows up in one recorded run — plus one nobody warned you about.

Moment	What it looked like	What it shows
The loop ran itself	24 tool calls in 27 requests, no human input	nobody scheduled these steps; the model chose each one
It checked a source	two web tables disagreed, so it went to FRED	verification happened because the model noticed — not by default
Errors became inputs	four blocked shell commands came back as <code>is_error</code> ; it read the file instead	“Observe” in ReAct — and “verified” now means “read carefully,” not “ran it”
One step had no tool	the party-by-year table came from memory	it happens to be right; nothing in the transcript lets you check
Context you did not write	a bundled data-viz skill turned “plot” into an interactive web page	the harness’s defaults are part of the prompt

Where a referee would stop you. The GDP series has provenance — a series ID, a source, a vintage. The party timeline has none. **Ask of every number an agent hands you: which tool produced this, and can I re-run it?**

The Same Task with Three Agents — a Design, Not a Run

The useful question is not “is this faster” but which steps are independent, and which one needs an adversary.



What fan-out actually buys

- the two *facts* are independent → genuinely parallel
- the series lands in data-fetcher’s context, **not** the orchestrator’s
- one job each, so “cite your source” is enforceable

What it does not buy

- the plot is one small step — splitting it adds turns, not speed
- three agents cost more tokens for the *same* numbers
- nothing here makes the causal claim more true

Only domain-reviewer **changes the answer** — and it cannot run in parallel: it needs something to attack.

What the Agent Wrote — and What No Agent Can Fix

Handed “rationalize,” this agent reframed the question as “Does the president’s party move GDP growth?” — and argued that it does not.

From its write-up: the gap in means (2.4% against 2.0%) comes from two years, 2009 and 2021, and **the medians are identical**; crediting each year to the president inaugurated that January is “a labeling choice, not evidence of causation”; Congress and the Fed set much of the policy; and **26 annual observations** cannot detect a partisan effect. It cites Blinder and Watson (2016). **[solid: the recorded transcript]**

Do not bank on the pushback. It still answered the question it was handed — party, not identification — chose the attribution rule itself, and left the timeline unsourced. A more agreeable model, or a subtler framing, gives you the rationalization.

And the limit. No number of agents fixes a leading prompt: a reviewer agent inherits your verb too — T5c’s Context Dilemma. The fix is upstream and free: ask “*is there a relationship, and what would identify it?*”

Bridge: Put the Loop on Your Own Machine

You have watched the loop on the wire: one real run, twenty-seven requests, and the places where a referee would stop you.

- A session is a list the harness re-sends; the agent is the loop, not a place
- You pay for every re-sent token — and for every number nobody sourced
- **Next (T2b):** put that loop on your own machine — install it, learn its files and commands, run your first session